

Intelligent System Development

Lecture 04 – Comparing CNN Implementations

Dinh Que Tran, Ph.D., Assoc. Prof.

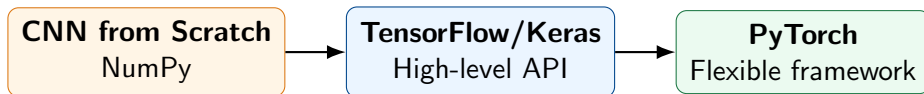
Posts and Telecommunications Institute of Technology
Hanoi, Vietnam
quetd@ptit.edu.vn

“The same learning idea can be implemented at different levels of abstraction.”

The purpose of this lecture is not simply to learn three CNN libraries.

We want to understand:

- 1 the **structure** of a CNN program;
- 2 the **function of each component**;
- 3 what the framework implements for us;
- 4 what remains the same across implementations.



Central Question

What is the same, and what is different?

We use two datasets to expose an important distinction.

Diabetes
Tabular data
 $x \in \mathbb{R}^d$

MNIST
Image data
 $X \in \mathbb{R}^{1 \times 28 \times 28}$

Diabetes:

Features $\rightarrow$ MLP $\rightarrow$ Prediction

MNIST:

Image $\rightarrow$ CNN $\rightarrow$ Prediction

Important

The model should match the structure of the data.

The Diabetes dataset is primarily a **tabular** classification problem.
For example:

$$x = [x_1, x_2, \dots, x_d]$$

contains numerical features.

There is normally no meaningful 2-D spatial neighborhood:

$$x_i \not\leftrightarrow x_{i+1}$$

in the same sense that neighboring pixels are related.

Therefore:

Diabetes $\rightarrow$ MLP is a natural baseline

while:

MNIST contains grayscale handwritten digit images.
Each sample can be represented as:

$$X \in \mathbb{R}^{1 \times 28 \times 28}.$$

The CNN can exploit local spatial structure.

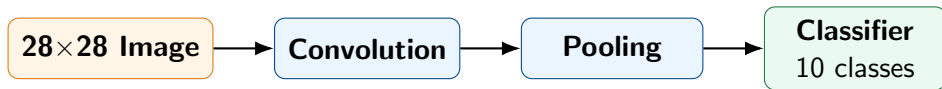
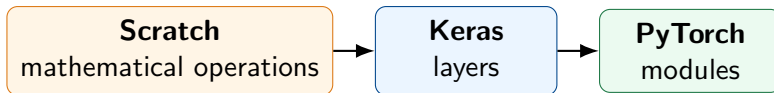


Image → Feature Extraction → Classification

Consider:

Conv $\rightarrow$ ReLU $\rightarrow$ Pool $\rightarrow$ Flatten $\rightarrow$ Dense.

This structure can be expressed as:



The **conceptual architecture is unchanged.**

What changes is:

Level of abstraction

Every implementation must express the same major ideas.

Concept	Purpose	Typical API
Convolution	local feature extraction	Conv2D / Conv2d
Activation	nonlinearity	ReLU
Pooling	spatial reduction	MaxPool
Flatten	tensor $\rightarrow$ vector	Flatten
Dense	classification	Dense / Linear
Loss	measure error	CrossEntropy
Optimizer	update parameters	Adam / SGD

Core Idea

The API names may change, but the computational role remains.

In the scratch implementation, each CNN component is written as an explicit Python function.

```
1 def convolution2d(x, kernel):
2     ...
3
4 def relu(x):
5     ...
6
7 def max_pool2d(x):
8     ...
9
10 def flatten(x):
11     ...
12
13 def linear(x, W, b):
14     ...
```

The functions are composed to form the forward pass:

$$x \xrightarrow{f_{\text{conv}}} x_1 \xrightarrow{f_{\text{relu}}} x_2 \xrightarrow{f_{\text{pool}}} x_3 \xrightarrow{f_{\text{flatten}}} x_4 \xrightarrow{f_{\text{linear}}} \hat{y}.$$

Therefore,

$$f(x) = f_{\text{linear}}(f_{\text{flatten}}(f_{\text{pool}}(f_{\text{relu}}(f_{\text{conv}}(x)))))$$

A simplified convolution operation can be expressed as:

```
1 def conv2d(x, kernel, stride=1):
2
3     H, W = x.shape
4     Kh, Kw = kernel.shape
5
6     Ho = (H - Kh) // stride + 1
7     Wo = (W - Kw) // stride + 1
8
9     output = np.zeros((Ho, Wo))
10
11     for i in range(Ho):
12         for j in range(Wo):
13
14             region = x[
15                 i*stride:i*stride+Kh,
16                 j*stride:j*stride+Kw
17             ]
18
19             output[i, j] = np.sum(
20                 region * kernel
21             )
22
23     return output
```

Here the student can directly see:

The complete structure is explicitly assembled.

```
1 def cnn_forward(x, params):
2
3     x = conv2d(x, params["K1"])
4     x = relu(x)
5
6     x = max_pool2d(x)
7
8     x = conv2d(x, params["K2"])
9     x = relu(x)
10
11    x = max_pool2d(x)
12
13    x = flatten(x)
14
15    x = linear(
16        x,
17        params["W"],
18        params["b"]
19    )
20
21    return x
```

The important observation is:

CNN = composition of functions

The student must also understand the learning cycle.

Forward → Loss → Gradient → Update

Conceptually:

```
1 for epoch in range(epochs):
2
3     for x, y in train_data:
4
5         logits = cnn_forward(x, params)
6
7         loss = cross_entropy(logits, y)
8
9         gradients = backward(
10             loss,
11             params
12         )
13
14         update(params, gradients)
```

The framework has not disappeared.

The student is simply implementing more of it.

Keras allows the same CNN structure to be expressed as reusable layers.

Conv → ReLU → Pool → Flatten → Dense

becomes:

```
1 model = tf.keras.Sequential([
2     tf.keras.layers.Conv2D(
3         32, 3, activation="relu"
4     ),
5     tf.keras.layers.MaxPooling2D(),
6
7     tf.keras.layers.Flatten(),
8
9     tf.keras.layers.Dense(
10        128, activation="relu"
11    ),
12
13    tf.keras.layers.Dense(10)
14 ])
```

What changed?

The convolution and pooling loops have disappeared from the student's code because they are encapsulated by layers.

Training is further abstracted.

```
1 model.compile(  
2     optimizer="adam",  
3     loss=tf.keras.losses.SparseCategoricalCrossentropy(  
4         from_logits=True  
5     ),  
6     metrics=["accuracy"]  
7 )  
8  
9 history = model.fit(  
10     x_train,  
11     y_train,  
12     epochs=10,  
13     validation_split=0.1  
14 )
```

Compare:

Scratch: manually implement training

versus:

Keras: model.fit()

PyTorch expresses the same CNN architecture using `nn.Module`.

```
1 class CNN(nn.Module):
2
3     def __init__(self):
4         super().__init__()
5
6         self.features = nn.Sequential(
7             nn.Conv2d(1, 32, 3, padding=1),
8             nn.ReLU(),
9             nn.MaxPool2d(2)
10        )
11
12        self.classifier = nn.Sequential(
13            nn.Flatten(),
14            nn.Linear(32 * 14 * 14, 10)
15        )
16
17    def forward(self, x):
18        x = self.features(x)
19        x = self.classifier(x)
20        return x
```

The conceptual structure is still:

Conv → ReLU → Pool → Flatten → Linear

PyTorch makes the data flow explicit.

```
1 def forward(self, x):  
2  
3     x = self.features(x)  
4  
5     x = self.classifier(x)  
6  
7     return x
```

Therefore:

`forward()` = definition of data flow

Compare this with Keras:

`Sequential(...)`

and Scratch:

`cnn_forward(x, params).`

PyTorch exposes the training cycle explicitly.

```
1 for images, labels in train_loader:
2
3     optimizer.zero_grad()
4
5     outputs = model(images)
6
7     loss = criterion(
8         outputs,
9         labels
10    )
11
12    loss.backward()
13
14    optimizer.step()
```

This corresponds directly to:

Zero Gradient → Forward → Loss → Backward → Update

Scratch	TensorFlow/Keras	PyTorch
Manual forward	Model definition	<code>nn.Module</code>
Manual loss	<code>compile()</code>	Loss function
Manual gradient	<code>fit()</code>	<code>loss.backward()</code>
Manual update	Framework optimizer	<code>optimizer.step()</code>
Manual training loop	High-level training	Explicit training loop

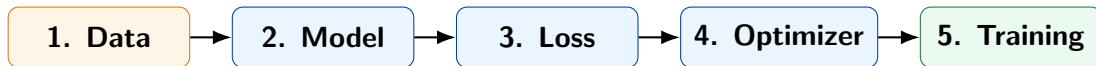
More abstraction $\longleftrightarrow$ less implementation detail

Function	Scratch	Keras	PyTorch
Convolution	<code>conv2d()</code>	Conv2D	<code>nn.Conv2d</code>
Activation	<code>relu()</code>	ReLU	<code>nn.ReLU</code>
Pooling	<code>max_pool2d()</code>	MaxPooling2D	MaxPool2d
Flatten	<code>flatten()</code>	Flatten	<code>nn.Flatten</code>
Dense	<code>linear()</code>	Dense	<code>nn.Linear</code>
Loss	manual	Keras loss	PyTorch loss
Gradient	manual	autodiff	autograd
Optimizer	manual	Keras optimizer	PyTorch optimizer

Main Lesson

Different names do not imply different machine-learning concepts.

Regardless of framework, students should identify these program components:



Then:

Evaluation → Prediction → Metrics

This structure applies to both:

Diabetes and MNIST.

For Diabetes:

$$X \in \mathbb{R}^{n \times d}.$$

A typical model is:

$$X \rightarrow \text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow \hat{y}$$

There is no natural convolution operation.

The three implementations can still be compared:

Scratch	Keras	PyTorch
<code>linear()</code>	<code>Dense()</code>	<code>nn.Linear</code>
<code>relu()</code>	<code>ReLU()</code>	<code>nn.ReLU</code>
manual loop	<code>fit()</code>	training loop

For MNIST:

$$X \in \mathbb{R}^{1 \times 28 \times 28}.$$

A simple architecture:

$28 \times 28 \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Pool} \rightarrow \text{Flatten} \rightarrow \text{Linear} \rightarrow 10$

The important point is that all three implementations must express this same architecture.

Suppose:

$$B \times 1 \times 28 \times 28.$$

After:

$$\text{Conv2d}(1, 32, 3, \text{padding} = 1)$$

we have:

$$B \times 32 \times 28 \times 28.$$

After:

$$\text{MaxPool}(2)$$

we have:

$$B \times 32 \times 14 \times 14$$

	Scratch	Keras	PyTorch
Convolution loops	Visible	Hidden	Hidden
Tensor operations	Visible	Mostly hidden	Partly visible
Autodiff	Manual / explicit	Automatic	Automatic
Training loop	Visible	Mostly hidden	Visible
Model structure	Explicit functions	Very compact	Explicit modules

Abstraction changes visibility, not the underlying mathematics.

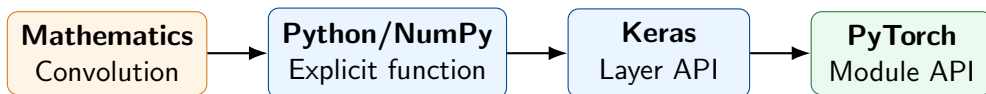
After implementing the three versions, students should be able to answer:

- ① What is the input representation?
- ② What does each layer compute?
- ③ What is the shape after every layer?
- ④ Which parameters are learnable?
- ⑤ Where is the forward pass?
- ⑥ Where is the loss computed?
- ⑦ Where is the gradient computed?
- ⑧ Where are parameters updated?

Do Not Memorize APIs

Understand the computational structure first.

The learning hierarchy is:



Understand → Implement → Abstract → Compare

One CNN – Three Implementations

Scratch	TensorFlow/Keras	PyTorch
Understand operations	Understand architecture	Understand architecture
Implement computations	Use high-level APIs	Use modular APIs
Manual training	High-level training	Explicit training
Maximum visibility	Maximum convenience	Flexibility + visibility

Students implement the same learning problem in three versions.

Part A – Diabetes

- implement an MLP from scratch;
- implement the same MLP using TensorFlow/Keras;
- implement the same MLP using PyTorch.

Part B – MNIST

- implement a simple CNN from scratch;
- implement the same CNN using TensorFlow/Keras;
- implement the same CNN using PyTorch.

Compare:

Structure + Components + Tensor Shapes + Training + Performance

Students submit a table such as:

Component	Scratch	Keras	PyTorch
Data loading			
Preprocessing			
Model definition			
Convolution			
Activation			
Pooling			
Flatten			
Dense layer			
Loss			
Gradient			
Optimizer			
Training loop			
Evaluation			
Prediction			

Main Question

Which framework hides the most implementation details? Which gives the programmer the

The implementation comparison should not stop at accuracy.

Students should record:

- number of parameters;
- training time;
- number of epochs;
- training loss;
- validation/test accuracy;
- precision, recall, F1;
- confusion matrix.

For a fair comparison:

Same Dataset + Same Split + Same Architecture + Comparable Hyperparameters

Data Representation



Mathematical Operation



Scratch Python



TensorFlow/Keras

Understand the data



Understand the model



Understand the functions



Understand the framework



Build intelligent systems